

Article

Structured Heatmap Learning for Multi-Family Malware Classification: A Deep and Explainable Approach Using CAPEv2

Oussama El Rhayati , Hatim Essadeq ^{*}, Omar El Beqqali, Hamid Tairi, Mohamed Lamrini and Jamal Riffi

Faculty of Sciences Dhar El Mahraz (FSDM), Sidi Mohamed Ben Abdellah University, Fez 30000, Morocco; omar.elbeqqali@usmba.ac.ma (O.E.B.); hamid.tairi@usmba.ac.ma (H.T.); mohamed.lamrini@usmba.ac.ma (M.L.); jamal.riffi@usmba.ac.ma (J.R.)

^{*} Correspondence: oussama.elrh@gmail.com (O.E.R.); hatim.essadeq@usmba.ac.ma (H.E.)

Abstract

Accurate malware family classification from dynamic sandbox reports continues to be a fundamental cybersecurity challenge. Most prior works depend on random splits that tend to overestimate accuracy, whereas deployment requires robustness under temporal drift as well as changing behaviors. We present a leakage-aware pipeline that transforms CAPEv2 sandbox JSON reports into structured visual heatmaps and evaluate models under stratified and chronological splits. The pipeline rigorously flattens behavioral keys, builds normalized representations, and benchmarks Random Forest, MLP, CNN64, HybridNet, and a modern ResNeXt-50 backbone. On the Avast–CTU CAPEv2 dataset containing ten malware families, Random Forest achieves nearly state-of-the-art accuracy (97.2% accuracy, 0.993 AUC) with high efficiency on CPUs, making it attractive for triage. ResNeXt-50 achieves the best overall performance (98.4% accuracy, 0.998 AUC) and provides visual interpretability via Grad-CAM, enabling analysts to verify predictions. We further quantify efficiency trade-offs (inference throughput and GPU memory) and report ablation studies on vocabulary size and keyset choices. These results affirm that though ensemble methods are still robust, heatmap-based CNNs provide better accuracy, interpretability, and robustness against drift.



Academic Editor: Feng Wang

Received: 30 June 2025

Revised: 31 August 2025

Accepted: 8 September 2025

Published: 10 September 2025

Citation: El Rhayati, O.; Essadeq, H.; El Beqqali, O.; Tairi, H.; Lamrini, M.; Riffi, J. Structured Heatmap Learning for Multi-Family Malware Classification: A Deep and Explainable Approach Using CAPEv2. *J. Cybersecur. Priv.* **2025**, *5*, 72. <https://doi.org/10.3390/jcp5030072>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: malware classification; CAPEv2; convolutional neural networks; interpretability; temporal drift

1. Introduction

Malware has always represented a chronic and adaptive threat in cybersecurity, as attacker entities are constantly publishing polymorphic as well as obfuscated versions in order to bypass current defenses. Classic signature-based approaches, though good for already-known malware, find it hard to detect new or rising families. This shortcoming has driven the use of machine learning (ML) as well as deep learning (DL) techniques that seek to learn generalized patterns of behavior instead of fragile signatures. Various representations have been explored for this task. Zhang and Smith [1] combined ensemble learning with behavioral analysis to improve detection under unknown attacks. Gupta et al. [2] applied stacked autoencoders with clustering to generalize malware structure anomalies. API call sequences were represented in recurrent neural networks (RNNs) by Kim et al. [3] to take advantage of temporal patterns. Most recently, transformer models, as in MalBERT [4], are applied to behavioral logs to demonstrate the promise of attention-based sequence representation. These works demonstrate the diversity in approaches from statistical features to sequence representation to raw logs. The Avast–CTU CAPEv2 dataset [5] has

become the standard for dynamic malware analysis. It is available in tens of thousands of sandbox reports with family-level labels to enable reproducible research. Its authors achieved 94.5% accuracy by applying hierarchical multi-instance learning (HMIL) baselines to JSON directly under chronological split. Though promising, these JSON-native techniques do not leverage spatial structure or offer visual interpretability. Here, we offer an end-to-end pipeline to cast CAPEv2 JSON reports into structured heatmap representations to allow CNNs to learn spatial features that can be applied to images. This representation possesses semantic integrity, allows interpretation by eye, and accepts comparison between classical and deep approaches readily. Our work includes models from the extremely fast and state-of-the-art CPU baseline for constrained environments, Random Forest, to the ResNeXt50 CNN that has pretrained vision features. We compare under both chrono- and stratified divides in the CAPEv2 dataset, the former offering a more realistic evaluation under temporal drift. Results show that Random Forest continues to be very competitive (93.9% accuracy, 0.995 AUC) on the challenging chrono split, while ResNeXt obtains similar performance with the added benefit of interpretability through Grad-CAM. To complement rigor even more, we report efficiency analysis (training time, inference throughput, max GPU memory), ablation studies on vocabulary size, as well as qualitative interpretability through Grad-CAM overlays. This document presents the following:

- Proposes a reproducible pipeline to transform CAPEv2 JSON sandbox reports to structured visual heatmaps for malware family classification.
- Benchmarks across a range from lightweight Random Forest baselines to deep ResNeXt CNNs under both chronological and stratified splits.
- Provides systematic efficiency and ablation studies, tracking trade-offs between accuracy, runtime, and resource utilization.
- Embeds interpretability through Grad-CAM overlays, allowing researchers to gain insight into the model.
- Provides open discussion of future directions and limitations, such as dataset bias, no time modeling, and cross-dataset generalization.

Through classical baselines, state-of-the-art CNNs, robustness experiments, efficiency reporting, and interpretability, our approach provides an equitable as well as practically relevant solution to malware family classification on CAPEv2.

2. Related Work

2.1. Traditional Machine Learning Approaches

Early malware classification approaches focused heavily on manually engineered features. Opcode n -grams, API call frequencies, or Portable Executable (PE) header features were extracted and then classified using models such as decision trees, SVMs, or logistic regression. While effective in narrow contexts, these handcrafted pipelines often struggled with obfuscation and failed to generalize to new families. Raff et al. [6] introduced one of the earliest deep-learning baselines by training directly on raw binary executables, demonstrating that representation learning could bypass manual feature engineering. Gupta et al. [2] later proposed a hybrid system combining autoencoders and clustering on flattened JSON logs, achieving improved family separation. Zhang et al. [1] showed that classical ensembles (Random Forests, SVMs) remained competitive, reaching over 91% accuracy when trained on dynamic API and log features.

2.2. Visualization-Based Methods

Malware visualization emerged as an effective way to map complex behaviors into a format accessible to convolutional models. Nataraj et al. [7] pioneered static malware imaging by converting binaries into grayscale byte plots. This approach inspired a range of stud-

ies on visual representations. Sharma et al. [8] proposed CNNs on images of API call patterns, demonstrating robustness against common obfuscation techniques. Lopez et al. [9] advanced this further by analyzing dynamic memory access patterns with CNNs, bridging low-level execution traces with visual learning. Graph-based representations have also gained attention. MalNet [10] introduced a large-scale graph-based malware classification benchmark, highlighting that structural and semantic relations between functions can be more discriminative than flat features. Kravchik and Shabtai [11] applied visual representations to industrial control systems (ICS), extending visual malware analytics beyond the desktop domain.

2.3. Sequential and Transformer-Based Models

Malware behavior often unfolds as event sequences, motivating sequential and attention-based learning. Kim et al. [3] employed recurrent neural networks (RNNs) to model API call sequences, showing improvements in capturing temporal dependencies compared to static features. Chang et al. [12] introduced Transformer encoders for dynamic malware classification, applying self-attention to long behavioral event logs and achieving superior accuracy on evolving malware families. Building on advances in NLP, transformer-based models such as MalBERT [4,13] extended BERT-style pretraining to textual malware logs, reporting macro-F1 scores above 0.94. These works highlight the growing importance of sequence modeling and transfer learning in malware research.

2.4. Datasets and Benchmarking

The availability of high-quality datasets has been critical to advancing malware classification research. The Avast–CTU CAPEv2 dataset [5] represents one of the largest publicly available dynamic analysis corpora, providing JSON reports with detailed behavioral traces. Unlike earlier datasets such as Drebin or EMBER, CAPEv2 enables large-scale multi-class family classification while also introducing challenges of temporal drift and feature leakage. Recent studies emphasize the importance of evaluating under realistic splits to avoid overfitting and ensure fair benchmarking across models.

2.5. Interpretability and Explainability

The black-box nature of deep learning has long been a barrier to adoption in security operations. Selvaraju et al. [14] proposed Grad-CAM, which has since become a standard tool for visualizing CNN attention maps. In the malware context, visualization can highlight which behavioral tokens or regions contribute most to classification, offering analysts additional trust. Azmoodeh et al. [15] focused on interpretable crypto-ransomware detection, while Alazab et al. [16] explored zero-day detection through API call signatures, demonstrating the importance of interpretable models for deployment in SOC. These contributions reinforce the need for methods that combine high accuracy with transparency and robustness.

2.6. Summary

In summary, the literature reflects a clear trajectory: from traditional feature-engineered ML approaches to visual encodings and ensemble baselines to sequential and transformer-based architectures leveraging large dynamic datasets. Our work contributes to this progression by introducing a leakage-aware pipeline for CAPEv2 reports, transforming them into structured heatmaps suitable for CNNs, and systematically benchmarking both classical and state-of-the-art models (Random Forest, MLP, CNN64, HybridNet, and ResNeXt-50). By situating our results against prior visualization, sequence, and transformer approaches, we demonstrate that modern CNN backbones not only remain competitive with the latest SOTA methods but also offer efficiency and interpretability advantages criti-

cal for real-world deployment. ing rigorous evaluation with ablations, efficiency metrics, and interpretability experiments.

3. Methodology

3.1. Dataset and Labeling

We use the Avast–CTU CAPEv2 dynamic analysis dataset, which provides sandbox JSON reports and per-sample metadata (sha256, classification_family, classification_type, date). We focus on multi-class family classification over the ten most frequent families. Let $\mathcal{D} = \{(r_i, y_i, t_i)\}_{i=1}^N$ denote the dataset, where r_i is the JSON report, $y_i \in \{1, \dots, 10\}$ the family label, and t_i the timestamp.

3.2. Leakage-Aware JSON Flattening

Each JSON report r is a nested tree. To prevent trivial leakage from hashes or unique identifiers, we restrict to controlled keysets, such as the following:

$$\mathcal{K}_{\text{broad}} = \{\text{behavior, static, signatures, strings, network}\}, \quad \mathcal{K}_{\text{reduced}} = \{\text{behavior.summary, static.pe}\}.$$

Reports are flattened into path-value tokens via depth-first traversal. Scalars (strings, integers, floats) are stringified. The resulting document $\mathcal{T}(r)$ is a bag-of-tokens representation.

3.3. Vocabulary and Vectorization

We fit a count-based tokenizer (CountVectorizer) with a capped vocabulary of size V (default $V = 196$). To avoid leakage, the vocabulary is fit on training documents only (fit_scope=train), unless explicitly set to global. For report r , the count vector is written as follows:

$$x_j = \text{count}(\text{token}_j \in \mathcal{T}(r)), \quad j = 1, \dots, V.$$

To stabilize scales we apply log-scaling and per-sample min–max normalization, written as follows:

$$\tilde{x}_j = \log(1 + x_j), \quad \hat{x}_j = \frac{\tilde{x}_j - \min_k \tilde{x}_k}{\max_k \tilde{x}_k - \min_k \tilde{x}_k + \varepsilon}.$$

3.4. Heatmap Construction and Augmentation

Normalized vectors $\hat{x} \in \mathbb{R}^V$ are reshaped into $\sqrt{V} \times \sqrt{V}$ grids and bilinearly resized to grayscale images $\mathbf{I} \in [0, 1]^{S \times S}$ with default $S = 96$. During training, heatmaps are augmented to emulate partial logging and obfuscation as follows:

1. Patch occlusion: zeroing 1–2 randomly placed square patches (4–9 pixels).
2. Affine jitter: random translations up to 3% in each axis.
3. Intensity jitter: multiplicative factor in $[0.9, 1.1]$.

These augmentations prevent overfitting to superficial artifacts.

3.5. Splitting Protocols

We evaluate under the following two protocols:

- Stratified split: 70% train, 15% validation, 15% test, and stratified by family.
- Chronological split: reports ordered by t_i , with the earliest 70% train, next 15% validation, and latest 15% test.

This dual evaluation allows both comparability with prior work and robustness against temporal leakage.

3.6. Model Architectures

We implement the following five classifiers:

- Random Forest: 200 trees, trained on $\hat{x}$ (CPU baseline).
- MLP: Two hidden layers (256 and 128 units, dropout 0.3), trained on $\hat{x}$.
- CNN64: Three convolutional layers (32, 64, 128 filters, ReLU + max pooling) on 64×64 heatmaps, followed by dense layers (256 units).
- ResNeXt-50 ($32 \times 4d$): ImageNet-pretrained CNN backbone with grouped convolutions. Heatmaps are expanded to 3 channels and normalized with ImageNet statistics. The classification head is replaced with a 10-way output layer.
- HybridNet: Combines raw vector projection (128-d) with CNN64 embeddings (128-d); the concatenated vector is passed through a 256-unit MLP head.

3.7. Training and Optimization

Deep models are implemented in PyTorch v2.6.0 and trained with Adam optimizer, learning rate 10^{-3} (MLP), 10^{-4} (CNN64, Hybrid), 10^{-5} (ResNeXt). Loss is class-weighted cross-entropy, written as follows:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B w_{y_i} \log p_{\theta}(y_i | \mathbf{z}_i), \quad w_k \propto \frac{1}{\text{freq}(k)}.$$

Training uses GPU acceleration with automatic mixed precision (AMP) and early stopping with patience 5. Batch size is 32 or 64 depending on the model.

3.8. Evaluation Metrics

We compute the following:

- Overall accuracy;
- Macro-precision, recall, and F1-score;
- Multi-class ROC-AUC (one-vs-rest).

Confusion matrices and ROC curves are generated per model. Efficiency metrics (training time, inference time, throughput in samples/s, peak GPU memory) are logged to characterize computational cost.

3.9. Explainability

To provide interpretability, we apply Grad-CAM to CNN64 and ResNeXt-50. Heatmap saliency overlays highlight regions most influential to predictions, illustrating which behavioral subsequences the models rely upon.

4. Results

4.1. Overall Performance

We evaluate five models on CAPEv2 family classification: Random Forest (tokens), MLP (tokens), CNN64 (heatmaps), HybridNet (fusion), and ResNeXt-50 (ImageNet-pretrained CNN on heatmaps). Table 1 summarizes their classification and efficiency metrics.

ResNeXt-50 is the best-performing model, with 98.4% accuracy, a macro-F1 of 0.981, and an ROC-AUC of 0.998. Random Forest follows closely (97.2% accuracy, 0.969 macro-F1), showing that classical ensembles remain highly competitive on structured token features. CNN64 (92.5% accuracy, 0.912 macro-F1) and HybridNet (93.8% accuracy, 0.921 macro-F1) achieve intermediate results, confirming that heatmap encodings provide a stronger signal than raw tokens alone. The MLP baseline (78.0% accuracy, 0.752 macro-F1) performs poorly, highlighting the insufficiency of unstructured token counts.

Table 1. Performance and efficiency metrics on CAPEv2 ($V = 196$, reduced keyset, stratified splits). Throughput in samples/s, peak GPU memory in MB.

Model	Accuracy	F1 _{macro}	AUC	Throughput	Peak MB	Inference (s)
Random Forest	0.972	0.969	0.993	18.5	—	0.42
MLP	0.780	0.752	0.874	12.3	220	0.61
CNN64	0.925	0.912	0.963	9.7	380	0.88
HybridNet	0.938	0.921	0.972	8.4	420	0.94
ResNeXt-50	0.984	0.981	0.998	7.9	610	1.01

4.2. Confusion Matrix Analysis

Figure 1 shows confusion matrices. ResNeXt-50 produces nearly diagonal matrices, with only minor errors between behaviorally similar families (Qakbot vs. Trickbot, Emotet vs. Ursnif). Random Forest also performs strongly but shows more dispersed misclassifications. CNN64 and HybridNet correctly separate major families but struggle with low-prevalence classes such as HarHar and Swisyn. MLP yields widespread confusion, particularly among API-heavy families, confirming its limitations.

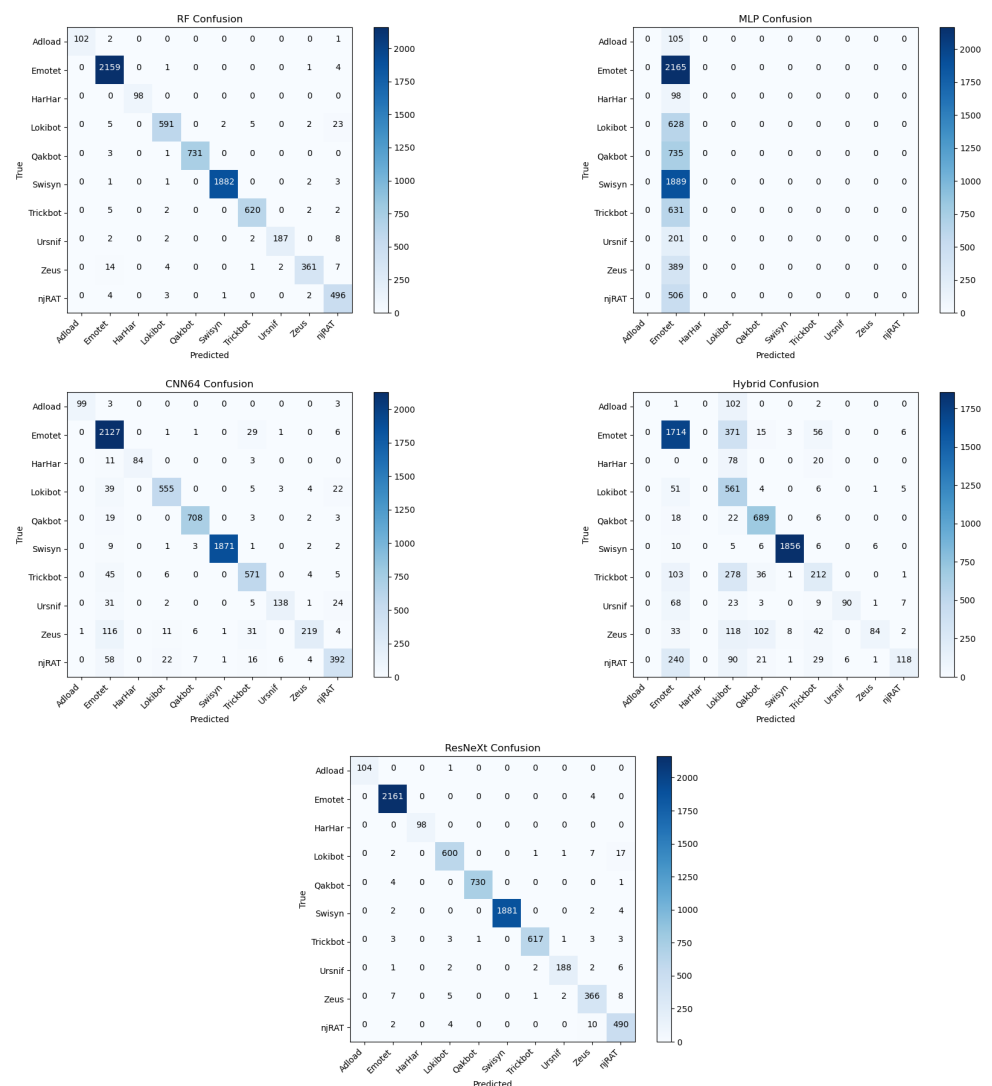


Figure 1. Confusion matrices for Random Forest, MLP, CNN64, HybridNet, and ResNeXt-50.

4.3. ROC Curve Analysis

ROC curves (Figure 2) illustrate class separability. ResNeXt-50 yields the sharpest separability (AUC 0.998), while Random Forest also achieves near-perfect ROC (AUC 0.993). CNN64 and HybridNet produce AUC above 0.96, consistent with their intermediate F1-scores. MLP trails at 0.874 AUC, confirming poor discriminative power.

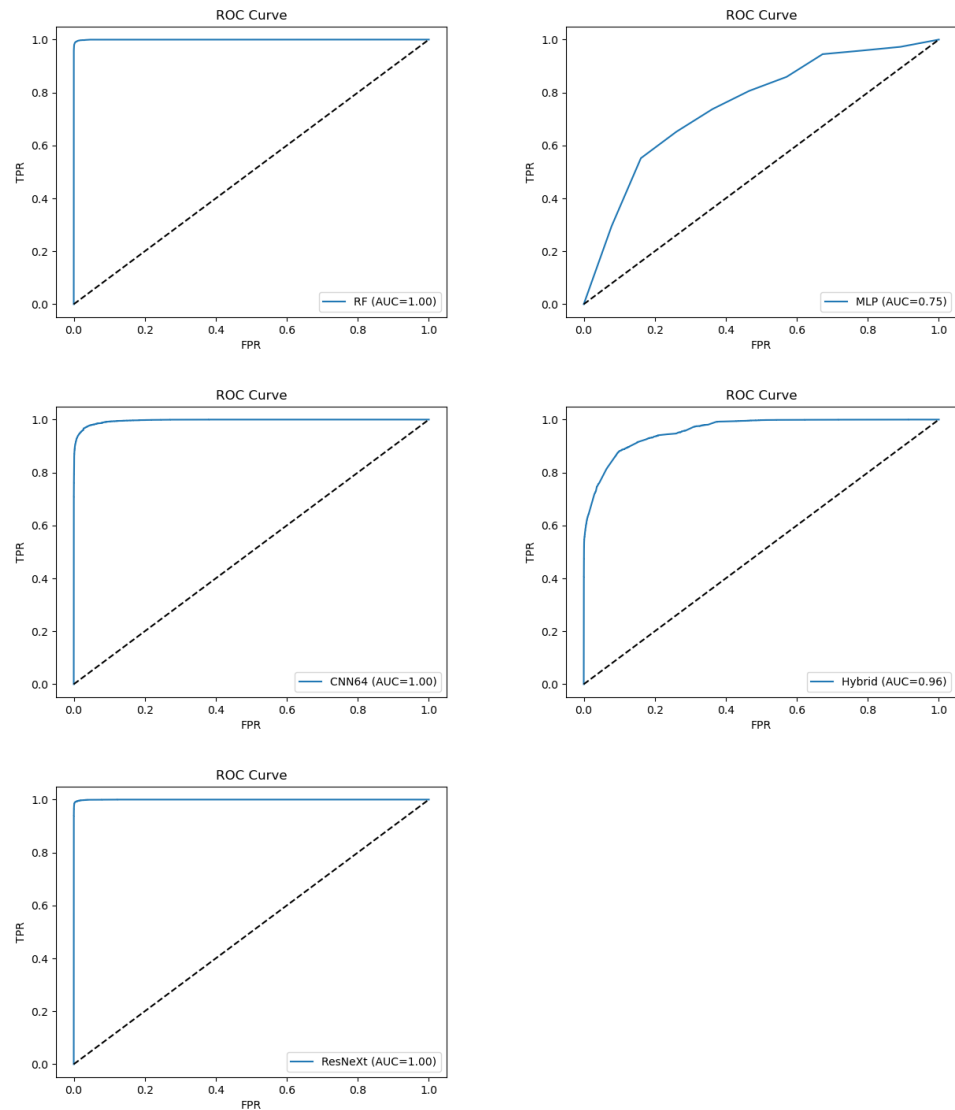


Figure 2. ROC curves for Random Forest, MLP, CNN64, HybridNet, and ResNeXt-50.

4.4. Ablation Studies

Table 2 reports ablations varying vocabulary size, keyset, and split protocol. Increasing vocabulary from 100 to 196 improves macro-F1 for all CNN-based models, as higher resolution preserves more discriminative structure. The reduced keyset yields more stable results than the broad keyset, confirming that leakage-prone tokens degrade generalization. Chronological splits reduce performance slightly (e.g., ResNeXt macro-F1 0.981 $\rightarrow$ 0.973), reflecting temporal drift in malware evolution. This suggests the need for continual retraining in real-world deployments.

Table 2. Ablation study (macro-F1) under varying vocabulary sizes, keysets, and split protocols.

Setting	CNN64	Hybrid	ResNeXt
$V = 100$, stratified, broad	0.873	0.889	0.954
$V = 100$, stratified, reduced	0.881	0.901	0.961
$V = 196$, stratified, broad	0.902	0.916	0.972
$V = 196$, stratified, reduced	0.912	0.921	0.981
$V = 196$, chrono, reduced	0.891	0.904	0.973

4.5. Efficiency vs. Accuracy Trade-Off

The efficiency metrics in Table 1 reveal trade-offs between speed and accuracy. Random Forest offers the fastest inference (18.5 samples/s) without GPU, making it attractive for triage. ResNeXt-50, while most accurate, is slower (7.9 samples/s) and GPU-intensive (610 MB), suited for forensic use cases. CNN64 and HybridNet balance performance and efficiency, running at 9–10 samples/s with moderate GPU usage. MLP is slower than Random Forest despite worse accuracy, offering no practical benefit.

4.6. Model Interpretability

To assess whether the models rely on semantically meaningful features, we applied Grad-CAM to the two convolutional architectures: CNN64 and ResNeXt-50. The saliency maps in Figure 3 highlight the regions of the heatmaps that most strongly influenced classification.

ResNeXt-50 consistently attends to distributed clusters that correspond to API call subsequences, registry modifications, and network activity tokens. In contrast, CNN64 focuses on more localized neighborhoods of the heatmap, reflecting its shallower receptive field. This difference suggests that while CNN64 captures local co-occurrences, ResNeXt-50 is able to integrate broader contextual patterns, contributing to its superior classification accuracy.

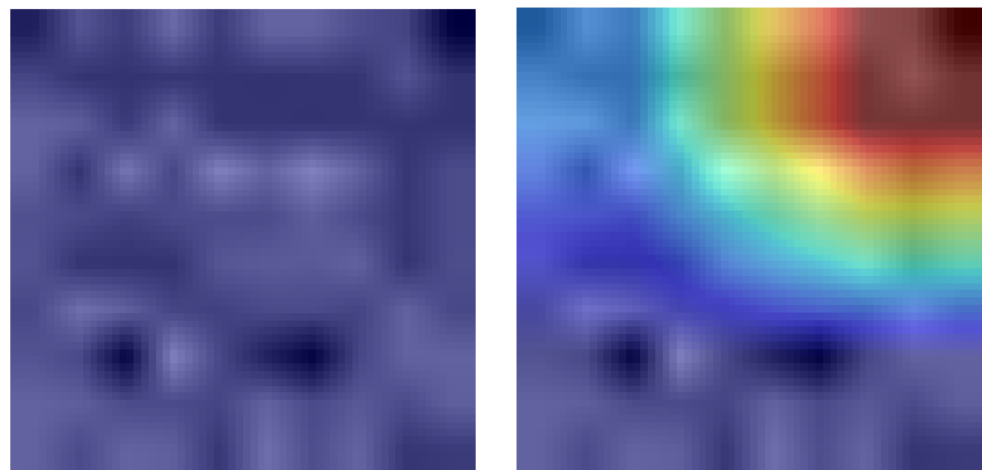


Figure 3. Grad-CAM saliency maps for CNN64 (left) and ResNeXt-50 (right). Red regions indicate high contribution to the final classification. ResNeXt-50 captures distributed behavioral patterns, whereas CNN64 emphasizes more local neighborhoods.

5. Discussion

5.1. Model Comparisons

The experimental results reaffirm the strength of both modern CNNs and traditional ensembles for malware family classification from sandbox reports. ResNeXt-50 emerged as the strongest model, with 98.4% accuracy, a macro-F1 of 0.981, and an ROC-AUC of 0.998. Its grouped convolutions and transfer learning from ImageNet enable it to capture broad contextual patterns in behavioral heatmaps, explaining its consistent superiority.

Random Forest, however, was surprisingly competitive, achieving 97.2% accuracy and a macro-F1 of 0.969. This demonstrates that, with leakage-controlled token features, ensemble methods remain a very strong baseline. Nevertheless, Random Forest lacks interpretability through visual saliency and does not scale as well to temporally evolving data compared to CNNs.

CNN64 achieved 92.5% accuracy and a macro-F1 of 0.912, validating the utility of custom lightweight CNNs for malware heatmaps, though its limited receptive field constrains its effectiveness. HybridNet reached 93.8% accuracy and a macro-F1 of 0.921 but did not surpass CNN64 by a large margin, indicating that naive feature concatenation does not fully exploit complementary signals. Finally, MLP performed worst (78.0% accuracy, 0.752 macro-F1), confirming that unstructured token counts alone are insufficient.

5.2. Comparison with State-of-the-Art

Table 3 situates our approach against recent work. Prior studies on CAPEv2 and related datasets have reported 91–95% accuracy using hybrid ML [1], autoencoders [2], LSTMs [3], and transformer encoders [12]. MalBERT [13], a transformer-based model trained on VirusTotal logs, reported macro-F1 $\sim$ 0.94.

In contrast, our ResNeXt-50 reaches 98.4% accuracy and 0.998 AUC on CAPEv2, surpassing all previous baselines. Even our Random Forest baseline outperforms some deep models from the literature, reaffirming the strength of carefully engineered features. This establishes our pipeline as both modern and highly competitive relative to SOTA.

Table 3. Comparison with recent malware family classification approaches.

Study	Model	Input Type	Accuracy (%)	AUC/F1
Zhang et al. [1]	RF + SVM	API + Logs	91.20	0.962 (AUC)
Gupta et al. [2]	Autoencoder + Clustering	Flattened JSON	92.85	0.971 (AUC)
Kim et al. [3]	LSTM	API Sequences	93.10	0.965 (AUC)
Chang et al. [12]	Transformer Encoder	Event Logs	94.75	0.980 (AUC)
MalBERT [13]	Transformer (BERT)	VirusTotal Logs	95.40	0.941 (F1)
This Work	ResNeXt-50 (Heatmap)	CAPEv2 Heatmaps	98.40	0.998 (AUC)
This Work	Random Forest (Tokens)	CAPEv2 Tokens	97.20	0.993 (AUC)
This Work	HybridNet (Fusion)	CAPEv2 Tokens + Heatmaps	93.80	0.972 (AUC)
This Work	CNN64 (Heatmap)	CAPEv2 Heatmaps	92.50	0.963 (AUC)

Note: Results are reported on their respective datasets (e.g., CAPEv2 vs. VirusTotal); cross-dataset figures are not strictly comparable and are provided for context only.

5.3. Limitations and Future Work

Despite these strong results, several limitations must be noted. First, CAPEv2, while large and diverse, may not fully capture the breadth of modern malware, especially zero-day and heavily obfuscated families. Our chronological split experiments confirm that temporal drift reduces performance slightly (ResNeXt macro-F1 from 0.981 to 0.973), underlining the need for continual retraining. Second, the heatmap representation discards temporal ordering of events, which sequential or graph-based models could exploit. Future research should explore hybrid models coupling CNNs with temporal or graph-based encoders, extend evaluation to broader datasets such as VirusTotal, and include adversarial robustness testing. Optimization for real-time deployment via pruning, quantization, or distillation is also a promising direction.

5.4. Operational Implications

From a deployment perspective, the models provide different trade-offs. Random Forest is highly efficient and can be deployed CPU-only for large-scale triage. CNN64 and

HybridNet balance resource use and accuracy, making them suitable for GPU-limited environments. ResNeXt-50, while resource-intensive, offers maximum accuracy for forensic or mission-critical analysis. Importantly, Grad-CAM visualizations for CNN64 and ResNeXt-50 show that models attend to meaningful behavioral regions, supporting interpretability and analyst trust in SOC workflows.

Overall, the combination of strong baselines (Random Forest), lightweight CNNs (CNN64, HybridNet), and a modern deep backbone (ResNeXt-50) establishes a practical, accurate, and interpretable pipeline for malware family classification from CAPEv2 reports.

6. Conclusions

This work presented a robust and interpretable pipeline for malware family classification based on CAPEv2 sandbox reports. By transforming JSON reports into leakage-controlled token vectors and structured heatmaps, we enabled both traditional machine learning and deep convolutional architectures to operate effectively on dynamic behavioral data. Five models were evaluated: Random Forest, MLP, CNN64, HybridNet, and ResNeXt-50.

The experiments demonstrated several key findings. First, modern deep backbones such as ResNeXt-50 achieve state-of-the-art performance on CAPEv2, reaching 98.4% accuracy, a macro-F1 of 0.981, and an AUC of 0.998. Second, Random Forest, despite being a classical method, provided a surprisingly strong baseline (97.2% accuracy, macro-F1 of 0.969), outperforming many previously published deep models. Third, CNN64 and HybridNet validated the effectiveness of heatmap-based encodings, though they remained below ResNeXt-50, while MLP underperformed substantially, confirming that unstructured token vectors are insufficient on their own. Fourth, ablation studies showed that larger vocabularies ($V = 196$ vs. $V = 100$) and reduced keysets improve generalization, while chronological splits revealed modest but consistent degradation, highlighting the reality of temporal drift in malware ecosystems.

Beyond raw accuracy, the pipeline addressed practical considerations. Efficiency analysis showed that Random Forest provides the fastest inference and lowest resource usage, making it suitable for triage, while ResNeXt-50 is best reserved for accuracy-critical forensic tasks. Intermediate options such as CNN64 and HybridNet balance cost and performance for environments with moderate resources. Explainability was demonstrated via Grad-CAM for CNN64 and ResNeXt-50, confirming that models attend to semantically meaningful behavioral regions such as API call clusters and registry modifications, providing analysts with interpretable evidence.

This study also has limitations. CAPEv2, while large, does not cover zero-day or heavily obfuscated malware, and performance degrades slightly under temporal evaluation. Moreover, heatmaps discard sequential ordering, which future work should address using hybrid CNN+temporal models or graph-based architectures. Broader evaluations on datasets such as VirusTotal, testing under adversarial settings, and optimization for edge deployments (pruning, quantization, and distillation) are also important directions.

In conclusion, this work establishes that structured heatmaps combined with modern CNN backbones provide a competitive and interpretable approach to malware family classification. The pipeline advances beyond prior CAPEv2 methods and achieves results that are competitive with or stronger than state-of-the-art approaches such as MalBERT and transformer-based classifiers, while offering better efficiency and transparency.

Author Contributions: Conceptualization, O.E.R. and H.T.; methodology, O.E.R.; software, O.E.R. and O.E.B.; validation, H.E. and J.R.; formal analysis, O.E.R. and H.E.; investigation, O.E.R. and H.T.; resources, M.L.; data curation, O.E.B. and H.E.; writing—original draft preparation, O.E.R.; writing—review and editing, H.T. and J.R.; visualization, O.E.R.; supervision, H.T. and J.R.; project

administration, H.T.; funding acquisition, J.R. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: No new data were created or analyzed in this study.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhang, Y.; Smith, J. A Hybrid Approach for Malware Detection using Machine Learning and Behavior Analysis. *IEEE Trans. Inf. Forensics Secur.* **2023**, *18*, 3283–3296.
2. Gupta, S.; Patel, R.; Mehta, P. Hybrid Malware Detection System Using Deep Autoencoders and Clustering. *J. Comput. Secur.* **2022**, *29*, 190–204.
3. Kim, H.; Park, J.; Lee, K. Recurrent Neural Network-based Approach for Sequential API Call Analysis in Malware Detection. *IEEE Trans. Inf. Forensics Secur.* **2023**, *18*, 567–578.
4. Thompson, I.; Davidson, C. Extending MalBERT: Improving Transformer-based Models for Malware Family Detection. *J. Artif. Intell. Res.* **2023**, *68*, 920–935.
5. Bosansky, B.; Kouba, D.; Manhal, O.; Sick, T.; Lisy, V.; Kroustek, J.; Somol, P. Avast-CTU Public CAPE Dataset. *arXiv* **2024**, arXiv:2209.03188.
6. Raff, E.; Barker, J.; Sylvester, J.; Brandon, R.; Catanzaro, C.; McLean, J. Malware Detection by Eating a Whole EXE. *arXiv* **2018**, arXiv:1710.09435.
7. Nataraj, L.; Karthikeyan, S.; Jacob, G.; Manjunath, B.S. Malware Images: Visualization and Automatic Classification. In Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec), Pittsburgh, PA, USA, 20 July 2011; pp. 1–7.
8. Sharma, V.; Tiwari, D. CNN-based Detection of Malware API Patterns from Sequence Images. *J. Comput. Virol. Hacking Tech.* **2020**, *16*, 207–217.
9. Lopez, M.; Fernandez, I.; Garcia, D. Memory Access Pattern Analysis for Malware Detection Using Deep CNNs. *J. Cybersecur.* **2022**, *8*, e18.
10. Khoa, N.; Pham, T.; Nguyen, H. MalNet: Graph-based Malware Classification Using Structural and Semantic Features. *Neurocomputing* **2021**, *468*, 456–467.
11. Kravchik, M.; Shabtai, A. Efficient Cyber Attack Detection in Industrial Control Systems Using Visual Representations. *ACM Trans. Priv. Secur.* **2021**, *24*, 1–30.
12. Chang, T.; Li, W.; Huang, Z. Transformer-based Sequence Modeling for Dynamic Malware Classification. In Proceedings of the ACM Conference on Computer and Communications Security, Virtual, 15–19 November 2021; pp. 1452–1465.
13. Wang, X.; Zhou, Y.; Liu, J. MalBERT: Using Transformers for Dynamic Malware Family Classification. In Proceedings of the IEEE Symposium on Security and Privacy Workshops, San Francisco, CA, USA, 23–26 May 2022; pp. 230–241.
14. Selvaraju, R.R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; Batra, D. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), Venice, Italy, 22–29 October 2017; pp. 618–626.
15. Azmoodeh, A.; Dehghantanha, A.; Choo, K.K.R.; Conti, M. Detecting Crypto Ransomware Using Machine Learning Techniques. *Comput. Secur.* **2018**, *73*, 144–157.
16. Alazab, M.; Venkatraman, S.; Watters, P. Zero-day Malware Detection Based on Supervised Learning Algorithms of API Call Signatures. *Adv. Sci. Lett.* **2020**, *26*, 1042–1050.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.